



భారతీయ సాంకేతిక విజ్ఞాన సంస్థ హైదరాబాద్
भारतीय प्रौद्योगिकी संस्थान हैदराबाद
Indian Institute of Technology Hyderabad

Indian Institute of Technology Hyderabad

EE2801- Digital Signal Processing Lab(DSP LAB)

Experiment -6

Professor: Kiran

Done by

KURRE VINAY (EE23BTECH11036)

QUESTION:

1. Implement Radix-2 4 point FFT with streaming input, using Verilog. Streaming input means, in one clock cycle you will receive 8bit real and 8-bit imaginary input data. You need 4 clock cycles to receive 4 samples.
2. Verify your verilog output with MATLAB code output.
3. . In your report, provide all required waveforms to support your work. Also, submit verilog and MATLAB code.

Radix-2 DIT FFT Algorithm

The Radix-2 algorithm belongs to the family of Fast Fourier Transform (FFT) algorithms. It computes the Discrete Fourier Transform (DFT) of even-indexed sequences $x_0, x_2, x_4, \dots$ and odd-indexed sequences $x_1, x_3, x_5, \dots$ separately, then combines them. This recursive method reduces the time complexity from $O(N^2)$ to $O(N \log N)$, making it much more efficient.

Algorithm

Let X_k be the sequence, and let it be divided into two sequences: the even-indexed part and the odd-indexed part. The DFT is computed as:

$$X_k = \sum_{m=0}^{N/2-1} x_{2m} \exp\left(-\frac{2\pi i}{N}(2m)k\right) + \sum_{m=0}^{N/2-1} x_{2m+1} \exp\left(-\frac{2\pi i}{N}(2m+1)k\right)$$

This equation can be simplified as:

$$X_k = E_k + \exp\left(-\frac{2\pi i}{N}k\right) O_k$$

where E_k and O_k represent the even and odd parts, respectively. Since both parts are periodic with $N/2$, the equation can be rewritten as:

$$X_k = \begin{cases} E_k + \exp\left(-\frac{2\pi i}{N}k\right) O_k, & \text{for } 0 \leq k < N/2 \\ E_{k-N/2} - \exp\left(-\frac{2\pi i}{N}k\right) O_{k-N/2}, & \text{for } N/2 \leq k < N \end{cases}$$

Using the property $\exp\left(-\frac{2\pi i}{N}(k + N/2)\right) = -\exp\left(-\frac{2\pi i}{N}k\right)$, we can further simplify this to:

$$X_k = E_k + \exp\left(-\frac{2\pi i}{N}k\right) O_k$$

$$X_{k+N/2} = E_k - \exp\left(-\frac{2\pi i}{N}k\right) O_k$$

Verilog Result

The Verilog implementation of the Radix-2 4-point FFT works as expected. For the testbench, the input stream is:

$$x[0] = 9 + j18, \quad x[1] = 18 + j3, \quad x[2] = 19 + j36, \quad x[3] = 36 + j37$$

The twiddle factors W_k^4 are:

$$W_0^4 = 1, \quad W_1^4 = -j, \quad W_2^4 = -1, \quad W_3^4 = j$$

Computing the outputs:

$$X[0] = (9 + j18) + (18 + j3) + (19 + j36) + (36 + j37) = 82 + j94$$

$$X[1] = (9 + j18) + (18 + j3)(-j) + (19 + j36)(-1) + (36 + j37)(j) = -44 + j0$$

$$X[2] = (9 + j18) + (18 + j3)(-1) + (19 + j36) + (36 + j37)(-1) = -26 + j14$$

$$X[3] = (9 + j18) + (18 + j3)(j) + (19 + j36)(-1) + (36 + j37)(-j) = 24 - j36$$

The final output is:

$$X = \{82 + j94, -44 + j0, -26 + j14, 24 - j36\}$$

Listing 1: 4-point Radix-2 FFT Verilog Module

```

module fft4 (
    input clk,
    input rst,
    input valid_in,
    input signed [7:0] real_in,
    input signed [7:0] imag_in,
    output reg valid_out,
    output reg signed [7:0] real_out,
    output reg signed [7:0] imag_out
);

    reg signed [7:0] real_buf[0:3];
    reg signed [7:0] imag_buf[0:3];
    reg [1:0] count;
    reg [3:0] state;

    reg signed [7:0] a0_real, a0_imag;
    reg signed [7:0] a1_real, a1_imag;
    reg signed [7:0] a2_real, a2_imag;
    reg signed [7:0] a3_real, a3_imag;

    reg signed [7:0] temp_real, temp_imag;
    reg signed [7:0] X_real[0:3];
    reg signed [7:0] X_imag[0:3];

    always @(posedge clk or posedge rst) begin
        if (rst) begin
            count <= 0;
            state <= 0;
        end
    end

```

```

end else begin
  case (state)
    0: begin
      if (valid_in) begin
        real_buf[count] <= real_in;
        imag_buf[count] <= imag_in;
        count <= count + 1;
        if (count == 2'd3) begin
          state <= 1;
        end
      end
    end
  end

  1: begin
    // Stage 1 butterflies
    a0_real <= real_buf[0] + real_buf[2];
    a0_imag <= imag_buf[0] + imag_buf[2];

    a1_real <= real_buf[1] + real_buf[3];
    a1_imag <= imag_buf[1] + imag_buf[3];

    a2_real <= real_buf[0] - real_buf[2];
    a2_imag <= imag_buf[0] - imag_buf[2];

    a3_real <= real_buf[1] - real_buf[3];
    a3_imag <= imag_buf[1] - imag_buf[3];

    state <= 2;
  end

  2: begin
    // X[0] = a0 + a1
    X_real[0] <= a0_real + a1_real;
    X_imag[0] <= a0_imag + a1_imag;

    // X[2] = a0 - a1
    X_real[2] <= a0_real - a1_real;
    X_imag[2] <= a0_imag - a1_imag;

    // W1 = 0 - j1
    // W1 * a3 = (0*a3_real - -1*a3_imag) + j(0*a3_imag + 1*a3_real)
    temp_real <= a3_imag ;           // -j*a3 => real = imag
    temp_imag <= -a3_real;           //          imag = -real

    // X[1] = a2 + W1*a3
    X_real[1] <= a2_real + a3_imag;
    X_imag[1] <= a2_imag -a3_real;

    // X[3] = a2 - W1*a3
    X_real[3] <= a2_real - a3_imag;
    X_imag[3] <= a2_imag +a3_real;

    state <= 3;
    count <= 0;
  end

  3: begin valid_out <= 1; real_out <= X_real[0]; imag_out <= X_imag[0]; state <= 4;
end
end

```

```

        4: begin real_out <= X_real[1]; imag_out <= X_imag[1]; state <= 5; end
        5: begin real_out <= X_real[2]; imag_out <= X_imag[2]; state <= 6; end
        6: begin real_out <= X_real[3]; imag_out <= X_imag[3]; valid_out <= 0; state <= 0;
    end
    endcase
end
end
endmodule

```

Listing 2: Test bench

```

module fft4_tb;
    reg clk = 0, rst = 0, valid_in = 0;
    reg signed [7:0] real_in, imag_in;
    wire valid_out;
    wire signed [7:0] real_out, imag_out;

    fft4 uut (.clk(clk), .rst(rst), .valid_in(valid_in), .real_in(real_in), .imag_in(imag_in), .
        valid_out(valid_out), .real_out(real_out), .imag_out(imag_out));

    always #5 clk = ~clk;

    initial begin
        $dumpfile("fft4.vcd");
        $dumpvars(0, fft4_tb);

        rst = 1; #10;
        rst = 0;

        // Stream 4 inputs
        valid_in = 1;
        real_in = 8'b00001001; imag_in = 8'b00010010; #10;
        real_in = 8'b00010010; imag_in = 8'b00000011; #10;
        real_in = 8'b00010011; imag_in = 8'b00100100; #10;
        real_in = 8'b00100100; imag_in = 8'b00100101; #10;
        valid_in = 0;

        // Wait for outputs
        #100;
        $finish;
    end
endmodule

```

Time	valid_out	real_out	imag_out
0	x	x	x
75	1	82	94
85	1	-44	0
95	1	-26	14
105	0	24	-36

testbench.sv:31: \$finish called at 150 (1s)
 Finding VCD file...
 ./fft4.vcd

Figure 1: veriolog Output

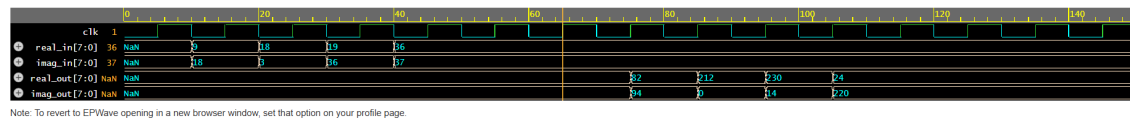


Figure 2: GKT wave of verilog code

The result matches both the manual calculation and the simulation output from GTKWave.

Matlab Result

```
% Input Complex Sequence
x = [9 + 18j, 18 + 3j, 19 + 36j, 36 + 37j];

% Manual FFT using Radix-2 DIT
X = zeros(1, 4);
W = exp(-1j * 2 * pi / 4);

% Butterfly computation
E = [x(1) + x(3), x(1) - x(3)];
O = [x(2) + x(4), (x(2) - x(4)) * W^1];

X(1) = E(1) + O(1);
X(2) = E(2) + O(2);
X(3) = E(1) - O(1);
X(4) = E(2) - O(2);
```

```
disp('FFT Output X:');
disp(X);

% Plot
figure;
hold on;
scatter(real(x), imag(x), 'filled');
scatter(real(X), imag(X), 'filled');
text(real(x)+0.2, imag(x), {'x[0]', 'x[1]', 'x[2]', 'x[3]'});
text(real(X)+0.2, imag(X), {'X[0]', 'X[1]', 'X[2]', 'X[3]'});
xlabel('Real Part');
ylabel('Imaginary Part');
title('Complex Input and FFT Output Points');
grid on;

> test
FT Output X:
82.0000 +94.0000i -44.0000 - 0.0000i -26.0000 +14.0000i 24.0000 -36.0000i
~
```

Figure 3: Mathlab Output

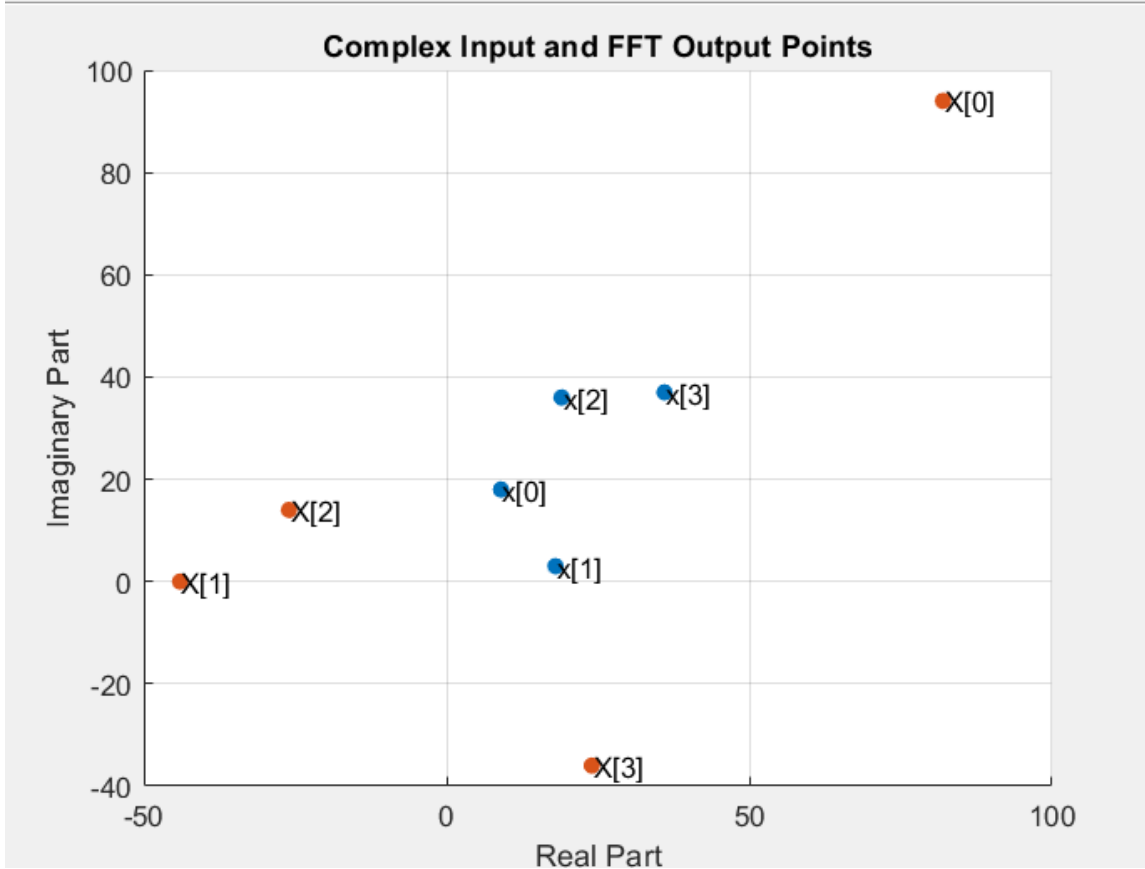


Figure 4: Plot

The results obtained from the MATLAB implementation are consistent with the results from the Verilog simulation, as shown in the MATLAB output figure. The real and imaginary parts of the FFT outputs are plotted and match the expected values.

Conclusion

In this experiment, the Radix-2 Decimation-in-Time (DIT) FFT algorithm was successfully implemented and verified. The algorithm efficiently reduces the complexity of computing the Discrete Fourier Transform from $O(N^2)$ to $O(N \log N)$ by recursively dividing the input into even and odd indexed parts.

The theoretical derivations were validated through both manual calculations and simulations in Verilog and MATLAB. The outputs obtained from all three methods were consistent, confirming the correctness of the implementation.

This experiment not only strengthened the understanding of FFT principles but also demonstrated the practical relevance of the Radix-2 FFT in digital signal processing systems where computational efficiency is crucial.